

2025 VIP Notebook

By: Maria Jothish

Please click  on the left to view the table of contents.

Topic: VIP Meeting 1/7
Notes: Went over last semester’s project
To Do (Completion Date): ☒ Obtain and utilize VIP Notebook for semester (1/8) ☒ Think about which team to be a part of for this semester (1/8)
Log: Email about not being able to attend next week class due to interview

Topic: VIP Meeting 1/14
Notes: Did not attend due to interview
To Do (Completion Date): ☒ Solidify team to be apart of for the semester (1/20) ☒ Fill out project plan for the semester (1/20)
Log: Just filled out project plan for the semester

Topic: VIP Meeting 1/21
Notes: - Research different algorithms - different research - continue research on what characteristics to collect - different types of connections - database, data integration - figure out - Create a minimum viable product - create a basic frontend - Subgrouping - - focus ai generation - one group: front end - one group: what data to collect, pairs vs groups, grouping algo - study buddy vs team pairing - Learn about Transactive Memory Systems - Testing: - Interactive survey to group of people that walk through your design and give feedback - UI - System Parts - testing output of AI
To Do (Completion Date): ☒ Meet with group for planning (1/24) ☒ Brainstorm ways to expand grouping algorithm (1/25) ☒ Expand grouping algorithm in some way (1/27)
Log: - Filled out lettucemeet but had conflicting appt. with meeting time. - Sub Meeting Notes: - Split into subgroups: - Algorithm: Maria, Tiffany, Rikitha - UI Design: Emma, Kathy - Backend/MVP: Kun Lin, Bryan - Goals for next week: 1. Extend the algorithm and conduct further research. Brainstorm improvements. 2. Create a mockup of 2 pages for UI design, streamline the UI process so everyone can be on the same page about usage. 3. Plan and get basic outline of data storage running - Ideas for algorithm improvement: 1. Add more characteristics to match survey 2. Add a feature that allows users to select group size. Currently the algorithm only returns pairs of students

- 3. Add weights to each characteristic based on the characteristic the student prioritizes over others (identified in survey). The current algorithm pairs a student with a student that has the most matching characteristics. By adding weights, even if student A has the most matching characteristics with student B, they will get paired with person C because they match person A on the characteristic they prioritized.
 - This will take some time and research. A bit complicated...
- My contribution to the project:
 - Updated grouping algorithm:
 - Added more characteristics our grouping algorithm can now consider when pairing students. It also aligns better with our survey.
 - New UI:



Topic: VIP Meeting 1/28

Notes:

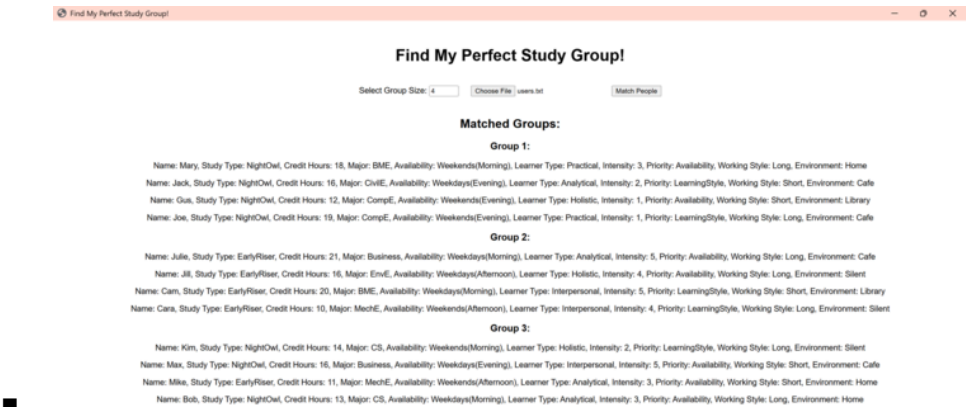
- Set in stone what group is doing
- Set member specific goals
- Need to figure out AI integration!!
- demo 2/11 !!
- notebook check 2/11 !!
- In class meeting notes:
 - Share projects on github
 - Develop grouping algo more
 - Research UI implementation
 - Revamp survey/matches page

To Do (Completion Date):

- ☒ Add project to github (2/4)
- ☒ Brainstorm idea for AI integration (Bryan is taking care of this aspect)
- ☒ Continue work on expanding grouping algorithm, add functionality that allows you to select your own group size (2/2)

Log:

- My contribution to the project:
 - Updated grouping algorithm:
 - Added feature that allows the users to select what size groups they want, so now instead of the algorithm only returning pairs of students, it returns groups of sizes 2 - 6.
 - Need to figure out what to do in case there are students left out in grouping due to not enough registered members
 - New UI:



- Github Upload:
 - Ran into the following issue which caused me to be unable to share the updated project on GitHub. Need to discuss with Bryan next class.

O

Topic: VIP Meeting 2/4

Notes:

- Guest Speaker Presentation:
 - createwithplay.com
 - LLM Providers: [groq](https://groq.com) (good for testing, but your data is not private), [ollama](https://ollama.com)/[Huggingface](https://huggingface.com)
 - [Gadget.dev](https://gadget.dev)
 - community platforms: [posthog](https://posthog.com), [hail](https://hail.sh)
- Next week:
 - **internal demo next week!** - show what we got for our prototype and talk about things you want to do/haven't got to doing yet.
 - Test plan (document): how can you test what you have done so far to make sure it is working.
 - high (ask for correctness of things, usability) vs low fidelity (ask if something is even necessary, concept, value proposition)
 - feasibility prototype (technical capabilities, potential constraints), functional, lives data prototypes (testing: functionality proven with data)
 - define pass/fail criteria
 - **Journals due!!**

To Do (Completion Date):

- ✓ Prepare for demo (2/10)
- ✓ Brainstorm Test Plan (2/10)
- ✓ Expand grouping algorithm, add weight functionality (Tiffany 2/11)
- ✓ Ensure journal is ready for submission (2/10)

Log:

- Demo Plan:
 - Each subgroup:
 - 1. Present what they have been working on
 - Expanding grouping algorithm
 - 2. Discuss future goals for further improvements
 - 3. Do brief overview of test plan
- Brainstorm Testing Plan:
 - High Fidelity
 - Grouping Algorithm Subgroup:
 - Manually write test cases that check for grouping algorithm accuracy
 - Ex: Write a test case that checks for the students with the most similar characteristics being grouped together.
 - Check if each group formed has the required number of students
 - Check algorithm efficiency as number of students using application increases
 - Test output using AI
 - UI group:
 - Send an interactive survey out to a group of people and have them walk through our design and give feedback

Topic: VIP Meeting 2/11

Notes:

- Send VIP Notebooks
- Test Plan submit this Friday

To Do (Completion Date):

- ✓ Complete and email test plan by Friday 2/14 @ 9 am
- ✓ Test Cases (2/18)

Log:

- Tiffany added weight functionality

Topic: VIP Meeting 2/18

- Log:
- Sick

My Contribution to the Project:

- Wrote 3 test cases to check functionality of grouping algorithm without weights
 - Checked for students with similar characteristics getting matched without weights applied
 - Checked for groups formed having the requested size
 - Checked scalability of algorithm by running it with 100 students

```
+ def test_1():
+     people = [
+         create_person(0, "Alice", "NightOwl", 13, "CS", "Weekdays(Morning)", "Analytical", 3, "Availability", "Long", "Home"),
+         create_person(0, "Bob", "NightOwl", 13, "CS", "Weekdays(Morning)", "Analytical", 3, "Availability", "Long", "Home"),
+         create_person(0, "Max", "NightOwl", 3, "CS", "Weekdays(Morning)", "Analytical", 3, "Availability", "Long", "Home"),
+         create_person(0, "Kim", "NightOwl", 3, "CS", "Weekdays(Morning)", "Analytical", 3, "Availability", "Long", "Home")
+     ]
+     weights_matrix = create_weights_matrix(len(people))
+     groups = match_people(people, 2, weights_matrix)
+
+     assert len(groups) == 2
+     assert len(groups[0]) == 2
+
+ # test 2: ensure groups of required size or less abd not more is being formed
+ def test_2():
+     people = [
+         create_person(i, f"Person{i}", "Alice", "NightOwl", 13, "CS", "Weekdays(Morning)", "Analytical", 3, "Availability", "Long", "Home")
+         for i in range(5)
+     ]
+     weights_matrix = create_weights_matrix(len(people))
+     groups = match_people(people, 3, weights_matrix)
+
+     assert len(groups) == 2
+     assert len(groups[0]) <= 3
+     assert len(groups[1]) <= 3
+
+ # test 3: trialing grouping algorithm performance capabilities
+ def test_3():
+     names = ["Alice", "Bob", "Max", "Kim", "Julie"]
+     people = [
+         create_person(i, f"Person{i}", random.choice(names), "NightOwl", random.randint(12, 21), "CS", "Weekdays(Morning)", "Analytical", random.randint(1, 5), "Availability", "Long", "Home")
+         for i in range(100)
+     ]
+     weights_matrix = create_weights_matrix(len(people))
+     groups = match_people(people, 4, weights_matrix)
+     assert len(groups) > 0
+     assert all(len(group) <= 4 for group in groups)
```

Topic: VIP Meeting 2/25

Notes:

- Highlight what u do in vip notebook
- Interview Qs Prof. Carl asks as an HM:
 - do you have experienced working on a team what was that like
 - any conflicts you experienced
 - if you could have your perfect project what would that look like
 - if you had a leader what was that like
- What did you find out from results and how do you incorporate that back into the prototype, have 3 slides for each group
- Need to focus on testing that improves algo/perhaps weight functionality
- Also need to evaluate how to determine "successful" group
- Need to share test results next week!
- Also share what you want to work on next 6 weeks based on past testing and experiences!
- dovetail
- concept testing

To Do (Completion Date):

- ☒ ~~Test Cases for algo with weights (3/4)~~
- ☒ ~~Prepare for test results presentation~~

- Log:
- Test Presentation Plan:
 - Each subgroup separately presents results

My Contribution to the Project:

- Added 3 more test cases to test grouping algorithm with weights
 - Checked for whether the application of weights is functioning correctly
 - Checked for compatibility of students being calculated correctly (i.e. higher weights + matching characteristics = higher compatibility score)
 - Checked if group with highest compatibility gets formed first.

grouping_algo_ui/test_weights.py

68

grouping_algo_ui/test_weights.py

Viewed

```
25 +
26 + # test 4: ensures that students with higher matching weights get a higher compatability
27 + def test_weight_effect():
28 +     student1 = {'index': 0, 'name': 'Bob', 'studyType': 'NightOwl', 'creditHours': 13, 'major': 'CS', 'availability': 'Weekdays(Morning)', 'learnerType': 'Analytical', 'intensity': 3,
29 +     'priority': 'Availability', 'workingStyle': 'Long', 'environment': 'Home'}
30 +     student2 = {'index': 1, 'name': 'Alice', 'studyType': 'EarlyRiser', 'creditHours': 17, 'major': 'MechE', 'availability': 'Weekdays(Afternoon)', 'learnerType': 'Practical', 'intensity': 4,
31 +     'priority': 'LearningStyle', 'workingStyle': 'Short', 'environment': 'Library'}
32 +
33 +     weights_low = [0.1] * 9
34 +     weights_high = [0.9] * 9
35 +
36 +     score_low = algorithm.calculate_compatibility(student1, student2, weights_low, weights_low)
37 +     score_high = algorithm.calculate_compatibility(student1, student2, weights_high, weights_high)
38 +
39 +     print(f"Compatibility score with low weights: {score_low}")
40 +     print(f"Compatibility score with high weights: {score_high}")
41 +
42 +     assert score_high > score_low, "Higher weights should result in a higher score"
43 +     print("Weight effect test passed!")
44 +
45 + # test 5: ensures group with highest computability is formed first
46 + def test_highest_compatibility_group_first():
47 +     student1 = {'index': 0, 'name': 'Bob', 'studyType': 'NightOwl', 'creditHours': 13, 'major': 'CS', 'availability': 'Weekdays(Morning)', 'learnerType': 'Analytical', 'intensity': 3,
48 +     'priority': 'Availability', 'workingStyle': 'Long', 'environment': 'Home'}
49 +     student2 = {'index': 1, 'name': 'Alice', 'studyType': 'NightOwl', 'creditHours': 13, 'major': 'CS', 'availability': 'Weekdays(Morning)', 'learnerType': 'Analytical', 'intensity': 3,
50 +     'priority': 'Availability', 'workingStyle': 'Long', 'environment': 'Home'}
51 +     student3 = {'index': 2, 'name': 'Max', 'studyType': 'EarlyRiser', 'creditHours': 17, 'major': 'MechE', 'availability': 'Weekdays(Afternoon)', 'learnerType': 'Practical', 'intensity': 4,
52 +     'priority': 'LearningStyle', 'workingStyle': 'Short', 'environment': 'Library'}
53 +     test_people = [student1, student2, student3]
54 +
55 +     # Bob and Alice prioritize studyType and major and Max prioritizes credithours and availability
56 +     weight_matrix = [
57 +         [0.5, 0.0, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
58 +         [0.5, 0.0, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
59 +         [0.0, 0.5, 0.0, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0]
```

Topic: VIP Meeting 3/4
Notes: - we can reuse slides for demo - we can explore new ideas, no need to incrementally improve
To Do (Completion Date): ☒ Work on AI Application (4/4)

Topic: VIP Meeting 3/11
Notes: - Expand UI - Do video of UI for demo - Have AI pinpoint patterns dummy data and update predetermined weights - UI for that that prints AI observations - generate synthetic data - UI Team Plan: 1. expand UI 2. Make it like an actual prototype, right now it is just clicks through the screens but it doesn't actually allow you to actually select anything 3. Work on demo prep: create a video walkthrough of prototype. - Algo: - AI Application to auto update weights - AI: - Bryan will link his AI results to our grouping algo so that we get the results of his AI analysis of the student's personality type to store as a characteristic of the student. - Professor Moon Feedback: - Our dataset needs to be much larger, current one is not enough.
To Do (Completion Date): ☒ Work on AI Application (4/4) ☒ Create larger dataset (3/18) ☒ Link dataset to AI model to return updated weights (4/1)

My Contribution to Project: - Created larger dataset using script: - Description: - This file generates a huge dataset of 50 groups, 3 students each. It also generates dummy student grades and finds the average grade of each group. The weights of the characteristics are all equal. This file is created to represent the data we collect after releasing the base grouping algorithm for a semester. After the semester is over we would like to see which groups performed better and identify any patterns in the characteristics of the students in those better performing groups. To do so we will link this data to generative AI. If the generative AI identifies a pattern correlating characteristics with better group performance it will display it and suggest a modification in the weights of the characteristics. The ultimate goal is to directly link the genAI results to the grouping algorithm and have it automatically update the weights of the characteristics. - For now this generated dataset purposefully gives students higher grades if they are grouped with students having the same learningstyle and/or availability to later see if generativeAI accurately identifies which grouped characteristics are associated with higher grades/averages. - Script:

```
updated_users_weights.txt  algorithm.py  test.py  test_weights.py  dataset_generation.py X  student_groups.json
dataset_generation.py > generate_students
14   or the characteristics.
15
16   For now this generated dataset purposefully gives students higher grades if they are grouped with students having
17   learningstyle and/or availability to later see if generativeAI accurately identifies which grouped characteristics
18   associated to higher grades/averages.
19   """
20   def generate_students(num_students=150):
21       study_types = ['EarlyRiser', 'NightOwl']
22       majors = ['CS', 'MechE', 'BME', 'CompE', 'Civile', 'EnvE', 'Business']
23       availabilities = ['Weekdays(Morning)', 'Weekdays(Afternoon)', 'Weekends(Morning)', 'Weekends(Afternoon)', 'Weekends(Evening)']
24       learner_types = ['Practical', 'Analytical', 'Interpersonal', 'Holistic']
25       priorities = ['LearningStyle', 'Availability']
26       working_styles = ['Short', 'Long']
27       environments = ['Library', 'Cafe', 'Silent', 'Home']
28
29       students = []
30       for i in range(num_students):
31           students.append({
32               'index': i,
33               'name': f'Student{i+1}',
34               'studyType': random.choice(study_types),
35               'creditHours': random.randint(10, 21),
36               'major': random.choice(majors),
37               'availability': random.choice(availabilities),
38               'learnerType': random.choice(learner_types),
39               'intensity': random.randint(1, 5),
40               'priority': random.choice(priorities),
41               'workingStyle': random.choice(working_styles),
42               'environment': random.choice(environments),
43               'grade': 0 # Will be replaced
44           })
45       return students
46
47   def create_groups(students, group_size=3):
48       # randomly creating groups (i.e. equal weights for all characterisitcs)
49       random.shuffle(students)
50       groups = []
51
52       for i in range(0, len(students) - group_size + 1, group_size):
53           group = students[i:i + group_size]
54
55           # check for same availability/learnertype
56           same_availability = all(s['availability'] == group[0]['availability'] for s in group)
57           same_learner_type = all(s['learnerType'] == group[0]['learnerType'] for s in group)
58
59           # Giving higher grade is students have same availability/learner type, else Lower
```

- Generated Dataset:

```
updated_users_weights.txt  algorithm.py  test.py  test_weights.py  dataset_generation.py  student_groups.json X
student_groups.json > ...
1  [
2      {
3          "members": [
4              {
5                  "index": 63,
6                  "name": "Student64",
7                  "studyType": "EarlyRiser",
8                  "creditHours": 18,
9                  "major": "CS",
10                 "availability": "Weekends(Afternoon)",
11                 "learnerType": "Holistic",
12                 "intensity": 4,
13                 "priority": "Availability",
14                 "workingStyle": "Short",
15                 "environment": "Silent",
16                 "grade": 70
17             },
18             {
19                 "index": 96,
20                 "name": "Student97",
21                 "studyType": "NightOwl",
22                 "creditHours": 13,
23                 "major": "MechE",
24                 "availability": "Weekends(Morning)",
25                 "learnerType": "Holistic",
26                 "intensity": 2,
27                 "priority": "LearningStyle",
28                 "workingStyle": "Long",
29                 "environment": "Library",
30                 "grade": 84
31             },
32             {
33                 "index": 113,
34                 "name": "Student114",
35                 "studyType": "EarlyRiser",
36                 "creditHours": 16,
37                 "major": "Business",
38                 "availability": "Weekdays(Morning)",
39                 "learnerType": "Practical",
40                 "intensity": 1,
41                 "priority": "Availability",
42                 "workingStyle": "Short",
43                 "environment": "Home",
44                 "grade": 65
45             }
46         ],
47         "average_grade": 73.0
48     }
49 ]
```

Topic: VIP Meeting 3/25 - Out sick

My Contribution to the Project:

- Linked generated dataset to a local AI model (Ollama-mistral) and it can identify characteristics that are correlated to better performance
- This file links student_groups.json to the local AI model and prompts it to identify patterns between characteristics availability, learner type, study style, intensity) and academic performance.
- Prompt:

```
# updated_users_weights.txt  algorithm.py  test.py  test_weights.py  dataset_generation.py  student_groups.json  dataset_ai_analysis.py X
dataset_ai_analysis.py > ...
1  import json
2  import subprocess
3
4  ...
5  This file links student_groups.json to the local AI model and prompts it to identify patterns between characteristics availability, learner type, study style, intensity) and academic performance.
6  ...
7
8  with open("student_groups.json", "r", encoding="utf-8") as f:
9      data = json.load(f)
10
11  prompt = f"""
12  You are analyzing a dataset of student study groups. Your goal is to:
13  1. Identify patterns between characteristics (availability, learner type, study style, intensity) and academic performance.
14  2. Determine if certain characteristics consistently correlate with **higher group grades**.
15  3. Suggest **weight adjustments** for the grouping algorithm based on these insights.
16
17  Here are **10 example groups** from the dataset:
18  {json.dumps(data[:10], indent=2)}
19
20  Please respond with a structured analysis:
21  - **High-Performing Group Patterns**: What characteristics lead to high grades?
22  - **Low-Performing Group Patterns**: What characteristics lead to low grades?
23  - **Recommended Weight Adjustments**: How should the algorithm prioritize characteristics based on this?
24  """
25
26  response = subprocess.run(
27      ["ollama", "run", "mistral", prompt],
28      stdout=subprocess.PIPE,
29      stderr=subprocess.PIPE,
30      encoding="utf-8"
31  )
32
33  print(response.stdout)
```

- Output:

```
1. High-Performing Group Patterns:
  - **Learner Type**: Interpersonal learners tend to perform better, with students labeled as "Interpersonal" having an average grade of 83.06 (excluding the outlier with a grade of 98). This suggests that group discussions, team projects, or collaborative learning environments may benefit these students.
  - **Environment**: Silent environments seem to be preferred by high-performing students, as they have an average grade of 77.62 (excluding the outlier with a grade of 98). This could indicate that concentration and focus play a significant role in academic success.
  - **Intensity**: Students with lower intensity levels (1 or 2) tend to perform better, having an average grade of 75.63. This might suggest that students who require less intense learning situations may thrive in an academic setting.

2. Low-Performing Group Patterns:
  - **Environment**: Students who prefer a more noisy environment, such as cafes, tend to perform poorly, with an average grade of 64.83. This could indicate that these students require a quieter space to focus and learn effectively.
  - **Learner Type**: Analytical learners seem to struggle, having an average grade of 70.25. They may benefit from more hands-on or visual learning experiences to help them better understand the material.
  - **Grade Intensity**: Students with high intensity levels (4 or 5) tend to perform poorly, having an average grade of 69.17. This could suggest that these students might be overwhelmed by the amount or difficulty of the coursework they are given.

3. Recommended Weight Adjustments:
  - The algorithm should prioritize learner type and environment when determining group assignments, assigning students who prefer silence to quiet spaces and prioritizing collaboration for interpersonal learners.
  - To help low-performing students, the algorithm could adjust weights to provide more opportunities for hands-on or visual learning experiences for analytical learners and reduce the intensity of coursework for students with high intensity levels. Additionally, the algorithm could identify noisy environments and provide alternatives for students who struggle in these settings.
```

Topic: VIP Meeting 4/1

Notes:

- pnambic presentation: 4/15!
- having 3-4 student personas
- next week dry run of demo

To Do (Completion Date):

- ☒ Work on Presentation for dry run (4/8)

Log:

- Finished slides

Topic: VIP Meeting 4/8

Notes:

- who is this person, grade, major, why are they coming to study buzz
- francis or antica
- notebook due next week

To Do (Completion Date):

- ☒ Need to update test cases to incorporate new personas (4/14)
- ☒ VIP Notebook Due
- ☒ Filter json so that only high performing groups get sent to ollama (4/15)
- ☒ Refine prompt sent to AI (4/15)
- ☒ Test its results (4/15)

Log:

- Update test cases

- Learned that AI model Ollama-mistral is hallucinating a bit....probably better to use a different one

My Contribution to the Project:

- Refined previous prompt sent to Ollama. Instead of having it identify the high performing groups from the generated dataset, I filtered it so only the high performing ones get sent. Now all ollama has to do is identify what characteristics are matching for all members in the group.

```
# Filtering json file to get high-performing groups
def get_high_performing_groups(groups, threshold=80):
    return [group for group in groups if group["average_grade"] >= threshold]

high_perf_groups = get_high_performing_groups(data)

prompt = f"""
```

```
prompt = f"""
You are analyzing student study groups. Each group contains multiple students. Each student has a set of characteristics and a final grade.

Your task is simple:

1. For each group listed below, compare the students **within that group only** and identify which of the following characteristics are **exactly the same for all members**:
- studyType
- creditHours
- major
- availability
- learnerType
- intensity
- workingStyle
- environment

2. Do not include any explanation, commentary, or summary – just output your analysis in the format below.

3. Use the following example as a guide:

Example Group:
{{
  "group_number": 19,
  "members": [
    {{
      "index": 98,
      "name": "Student99",
      "studyType": "NightOwl",
      "creditHours": 17,
      "major": "MechE",
      "availability": "Weekdays(Morning)",
      "learnerType": "Analytical",
      "intensity": 5,
      "priority": "LearningStyle",
      "workingStyle": "Long",
      "environment": "Cafe",
      "grade": 96
    }},
    {{
      "index": 51,
      "name": "Student52",
      "studyType": "NightOwl",
      "creditHours": 13,
      "major": "CS",
      "availability": "Weekends(Morning)",
      "learnerType": "Analytical",
      "intensity": 3,
      "priority": "Availability",
```

New Output:

```
C:\Users\Maria\Documents\Desktop\College\Spring_2025\VIP\text-demo-vip\grouping_algo_ui>python dataset_ai_analysis.py
High-performing groups: 2
High-Performing Groups Analysis:

Group 18:
- Average Grade: 90.66666666666667
- Shared Characteristics: [studyType, availability, learnerType, environment]

Group 19:
- Average Grade: 90.33333333333333
- Shared Characteristics: [studyType, learnerType]
```

- I also wrote a way to identify matching characteristics correlated to higher performance in python just to compare and test the results of the AI model.

```
C:\Users\Maria\Documents\Desktop\College\Spring_2025\VIP\text-demo-vip\grouping_algo_ui>python dataset_py_analysis.py
---
High-Performing Groups Analysis:

Group 18:
- Average Grade: 90.67
- Shared Characteristics: ['availability', 'workingStyle']

Group 19:
- Average Grade: 90.33
- Shared Characteristics: ['studyType', 'learnerType']
```